



APSCO

---

## LINKU Project

### CAMERA SYSTEM WORKING DOCUMENT

#### "BLENDER SIMULATION REALISM VALIDATION FOR TETHER IMAGING"

October 9, 2026

---

## Blender Simulation Realism Validation for Tether Imaging

---

Prepared by:

<sup>1</sup> Pontificia Universidad Catolica del Peru (PUCP)

**Author(s):** Jose Sifuentes<sup>1</sup>

**Status:** Working draft, experiments pending

**Issue:** 1

**Revision:** 0



## Abstract

This working document establishes the methodology to evaluate how realistically Blender (Cycles renderer) can reproduce the visual conditions relevant to LINKU tether imaging, and to what extent that realism supports the use of Blender-rendered images as a stand-in for imagery that cannot yet be captured physically, namely the tether observed under orbital illumination in vacuum.

The validation strategy follows the precedent set by SISPO (Space Imaging Simulator for Proximity Operations), a peer-reviewed Blender-Cycles-based simulator validated by direct pixel-level comparison against real spacecraft camera imagery. The same principle is applied here at a smaller scale: a Blender scene replicating the laboratory dark enclosure and tether proxy is rendered and compared against real photographs taken in that same enclosure, using the project's own reconstruction pipeline (segmentation, endpoint detection, path tracking) as the comparison instrument rather than only visual inspection. Establishing agreement at this laboratory scale, where a real reference photograph exists, is the necessary first step before trusting a second Blender scene configured for orbital illumination, for which no real reference photograph can exist yet.

This document is a working draft. The literature review and methodology are complete. A preliminary self-consistency check (Section 3.1) has been run within Blender alone, confirming the reconstruction pipeline recovers a known synthetic displacement to within 1 cm; the core experiment, the matched real-photograph-versus-render comparison, remains pending and is marked accordingly throughout the text.

# Contents

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>Introduction</b>                                                    | <b>5</b>  |
| <b>1 Background and Literature Review</b>                              | <b>6</b>  |
| 1.1 Physically based rendering realism                                 | 6         |
| 1.1.1 Other published Blender-based space imaging tools                | 6         |
| 1.2 Characterizing solar illumination                                  | 7         |
| 1.2.1 Spectral color                                                   | 7         |
| 1.2.2 Irradiance                                                       | 7         |
| 1.3 Where the illumination parameters come from                        | 8         |
| 1.4 Validating rendered realism against real images                    | 8         |
| <b>2 Validation Methodology</b>                                        | <b>9</b>  |
| 2.1 Scope                                                              | 9         |
| 2.2 The core problem: no ground truth exists for the orbital condition | 9         |
| 2.3 Comparison instrument: the reconstruction pipeline itself          | 9         |
| 2.4 Independent variables to control                                   | 10        |
| <b>3 Experimental Setup</b>                                            | <b>12</b> |
| 3.1 LINKU structure replica and preliminary self-consistency check     | 12        |
| 3.2 Blender scene construction                                         | 14        |
| 3.2.1 Chosen camera sensor and intrinsics                              | 14        |
| 3.2.2 Near-term substitution: laboratory work proceeds with the IMX477 | 18        |
| 3.2.3 Blender's own configuration reference for the Sun light          | 18        |
| 3.3 Laboratory photograph capture                                      | 18        |
| 3.4 Matched-pair generation                                            | 19        |
| 3.5 Orbital-condition render generation                                | 19        |
| <b>4 Evaluation and Reporting Plan</b>                                 | <b>20</b> |
| 4.1 Reporting format                                                   | 20        |
| 4.2 Interpretation criteria                                            | 20        |
| 4.3 Outcome paths                                                      | 20        |
| <b>5 Conclusions and Status</b>                                        | <b>21</b> |
| 5.1 Current status                                                     | 21        |
| 5.2 Next steps                                                         | 21        |

# List of Figures

|     |                                                                                                      |    |
|-----|------------------------------------------------------------------------------------------------------|----|
| 2.1 | Two-tier validation strategy: laboratory agreement first, orbital extrapolation second               | 10 |
| 3.1 | Blender replica of the LINKU deployment structure, general view . . . . .                            | 12 |
| 3.2 | Tether proxy as seen from each stereo camera inside the Blender LINKU structure<br>replica . . . . . | 13 |
| 3.3 | Depth Map tab output for the two known-displacement configurations . . . . .                         | 13 |

# List of Tables

|     |                                                                                                       |    |
|-----|-------------------------------------------------------------------------------------------------------|----|
| 1.1 | Published tools that use Blender as the rendering core for space-imaging simulation.                  | 7  |
| 3.1 | Optical comparison of all candidate sensor/lens configurations . . . . .                              | 15 |
| 3.2 | Chosen camera intrinsics, Arducam/Raspberry Pi 12MP IMX708 Standard, fixed focus (SKU B0308). . . . . | 17 |

# Introduction

The LINKU stereo vision system has been developed and validated using a laboratory dark enclosure that reproduces the low-texture, high-contrast optical conditions expected during orbital tether observation, as described in the Camera System Report. That validation is limited to what can physically be built and photographed on the ground. It cannot directly test the actual orbital environment: vacuum, unfiltered solar illumination, and the absence of any enclosing structure.

Blender, and specifically its Cycles physically based renderer, offers a way to extend testing beyond the laboratory by generating synthetic images of the tether under conditions that are difficult or impossible to reproduce physically, most importantly orbital solar illumination. However, a synthetic image is only useful as evidence if there is a documented basis for trusting that it behaves like a real photograph would, at least with respect to the properties the reconstruction pipeline actually depends on: radiometric contrast, edge sharpness, and background noise characteristics.

This document defines how that trust is established. The approach does not attempt to prove that a Blender render "looks like" a photograph in a general sense. Instead, it proposes a concrete, two-step validation:

1. Build a Blender scene that replicates the laboratory dark enclosure and tether proxy as closely as possible, render it, and compare the render against a real photograph of the same physical configuration, using the project's own reconstruction pipeline as the comparison instrument rather than visual inspection alone.
2. Only after that laboratory-scale agreement is demonstrated, reconfigure the same Blender scene to represent orbital illumination (vacuum, AM0 solar spectrum, no enclosure) and treat its output as a evidence-supported extrapolation, not an unverified assumption.

This structure follows the precedent of SISPO, a peer-reviewed Blender-Cycles-based space imaging simulator that established its own credibility by comparing renders directly against real spacecraft camera images [1]. The tether observation problem addressed here is smaller in scope, but the underlying validation logic is the same: a simulator is only as trustworthy as the evidence that it matches reality where reality can still be checked.

This document is organized as follows:

- Chapter 1 reviews the relevant literature on physically based rendering realism and on the characterization of solar illumination for space applications.
- Chapter 2 defines the validation methodology, including the two-tier laboratory-then-orbital strategy and the metrics used for comparison.
- Chapter 3 defines the experimental setup required to execute the methodology.
- Chapter 4 defines how results will be evaluated and reported once the experiments are executed.

This document is a working draft. The literature review (Chapter 1) and methodology (Chapter 2) are complete. The experimental setup (Chapter 3) and results (Chapter 4) are pending execution of the Blender scene construction and the laboratory photo capture session.

# Chapter 1

## Background and Literature Review

### 1.1 Physically based rendering realism

Blender's Cycles engine is a path tracing renderer that simulates the physical transport of light energy through a scene, including scattering, reflection, and refraction, rather than approximating these effects with non-physical shading tricks. Because of this, Cycles renders can be difficult to distinguish from real photographs when scene geometry, materials, and illumination are defined with physically accurate values rather than visually adjusted ones [2].

The relevant precedent for this document is SISPO (Space Imaging Simulator for Proximity Operations), a Blender-Cycles-based simulator developed for small solar system body fly-by and terrestrial planet surface mission simulation [1]. SISPO is directly relevant for two reasons:

- It validates realism the same way this document proposes to: by comparing rendered output directly against real spacecraft camera imagery, in that case the AMICA instrument images of asteroid Itokawa taken by the Hayabusa spacecraft, rather than by subjective visual inspection.
- Beyond scene lighting and geometry, it models camera-level realism explicitly: dark-current sensor noise drawn from a folded normal distribution, tangential and sagittal astigmatism and comatic aberration through dedicated point-spread functions, and lens distortion through a Brown-Conrady radial and tangential distortion model.

This indicates that scene-level realism (correct geometry and lighting) and camera-level realism (sensor noise and optical aberrations) are treated as two separate concerns that both need to be addressed for a render to be trustworthy as a proxy for a real photograph. Section 2.1 defines which of these two concerns this document addresses first, and which are deferred as future refinements.

#### 1.1.1 Other published Blender-based space imaging tools

SISPO is not an isolated case. Table 1.1 summarizes three other tools that use Blender as the rendering core for space-imaging applications, found while searching for this document, each with a different validation strategy.

A separate preprint surveying open-source tools for lunar synthetic imagery lists Blender alongside Unity as a general-purpose option, contrasted with space-specific commercial tools (PANGU, SurRender) and with CORTO itself, described there as "used to model space imaging scenarios with high realism," validated against real mission data [5]. This is treated as a secondary, not yet peer-reviewed source (stated as such in the preprint itself) and is cited only to point to CORTO, not as an independent methodology reference.

The common pattern across all three primary tools is the same one this document already follows:

Table 1.1: Published tools that use Blender as the rendering core for space-imaging simulation.

| Tool            | Origin                          | Validation / realism approach                                                                                                                                                                                            |
|-----------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SISPO [1]       | Univ. of Tartu / Aalto Univ.    | Pixel-level comparison against real Hayabusa/AMICA imagery of asteroid Itokawa; explicit camera noise and lens-aberration models.                                                                                        |
| CORTO [3]       | DART Lab, Politecnico di Milano | Built on Blender 4.0; used operationally for CubeSat and lunar mission image generation and closed-loop GNC testing.                                                                                                     |
| BlenderProc [4] | German Aerospace Center (DLR)   | Not space-specific; general-purpose photorealistic training-data pipeline. Included here because it is the most widely adopted procedural-scripting pattern for Blender-as-a-library, which CORTO and SISPO both follow. |

Blender/Cycles for physically based rendering, plus a validation step against something real, either real mission imagery (SISPO) or operational deployment in an existing lab pipeline (CORTO). No tool among those found publishes an exact numeric Sun-strength or camera-exposure value that could be copied directly; each defines its own scene-specific calibration, which is consistent with this document’s own approach of deriving values from first principles rather than from a single external reference number.

## 1.2 Characterizing solar illumination

A physically based renderer only produces a physically meaningful image if its light sources are configured with real radiometric and spectral values, not with values chosen to "look right" on a monitor. Two properties of the sun matter for this purpose: its spectral color, and its irradiance.

### 1.2.1 Spectral color

The integral of the measured extraterrestrial solar spectrum matches the integral of a blackbody spectrum at 5772 K [6]. This is the specific figure to use, rather than other round numbers sometimes seen in informal sources, since it is the value stated directly in the SOLAR-ISS reference spectrum paper based on SOLAR/SOLSPEC space-based measurements. Blender’s Sun light object supports a Blackbody shader node that takes a color temperature directly and produces the corresponding spectral color, so this value can be used directly rather than approximated with a manually picked color. A broader review of solar spectral characterization methods and their mutual agreement is given by Thuillier et al. [7].

For applications requiring higher spectral fidelity than a blackbody approximation, dedicated reference spectra exist. The SOLAR-ISS reference spectrum, derived from the SOLAR/SOLSPEC instrument aboard the International Space Station, provides measured solar spectral irradiance from 165 to 3000 nm at high spectral resolution [6]. This is a more rigorous reference than a blackbody approximation but is unlikely to be necessary for this validation, since the reconstruction pipeline operates on broadband intensity images rather than spectral data; the blackbody approximation is expected to be sufficient and is recommended as the default.

### 1.2.2 Irradiance

Blender’s Sun light strength in Cycles is expressed directly in units of irradiance, watts per square meter ( $\text{W}/\text{m}^2$ ), which is the same unit used to report solar irradiance [2]. This makes the value directly settable rather than requiring unit conversion.

Two different irradiance values are relevant depending on which condition is being simulated:

- **Ground-level, atmosphere-attenuated illumination:** commonly cited around  $1050 \text{ W}/\text{m}^2$  for direct sunlight after atmospheric absorption and scattering. This is the value used in most Blender outdoor-lighting tutorials, because most Blender scenes represent terrestrial, not orbital, conditions.

- **Extraterrestrial illumination (AM0, Air Mass Zero):** the internationally recommended solar constant (IAU, 2015) is approximately  $1361 \text{ W/m}^2$ . This is the traditional engineering reference value, used in standards such as ASTM E490 for spacecraft and photovoltaic component testing, and is recommended as the default for this document. A more recent reprocessing of SOLAR/SOLSPEC space-based measurements reports a total solar irradiance of  $1372.3 \pm 16.9 \text{ W/m}^2$ , stated explicitly as  $11 \text{ W/m}^2$  above the IAU 2015 value [6]. The two values differ by less than 1%, and either is defensible; this document adopts  $1361 \text{ W/m}^2$  because it is the more widely referenced engineering value, and notes  $1372.3 \text{ W/m}^2$  as an available refinement if higher precision is later judged necessary.

The orbital-condition Blender scene described in this document must use one of the two AM0 values above, not the ground-level value. This distinction is easy to overlook because most publicly available Blender tutorials and default scene templates assume a terrestrial, atmosphere-filtered Sun.

The ASTM E490 AM0 spectral table has since been obtained and integrated directly. That work, together with the conversion of the solar constant into the photometric units a lighting specialist works in, is reported in the companion deliverable *Dark-Room Solar Illumination Specification for Tether Imaging*, which supersedes the illumination material formerly carried in this document.

### 1.3 Where the illumination parameters come from

Two distinct light configurations are needed for the two renders of this document, and they are sourced differently.

- **Orbital-condition render.** Sun strength  $1361 \text{ W/m}^2$  (AM0) and a blackbody colour of  $5772 \text{ K}$ , as established above. These are nominal constants and require no measurement.
- **Laboratory-condition render.** Sun strength, colour temperature, lamp position and aim, and the light's angular size, which is the parameter controlling shadow softness, all come from the physical dark-room session. The specification for that session, its measurement protocol and the record sheet that carries the values back into this scene are in the companion deliverable *Dark-Room Solar Illumination Specification for Tether Imaging*. Until that session has been run, the laboratory-condition light in this document remains a placeholder.

Pending: replace the placeholder laboratory-condition light values with the measured illuminance, colour temperature, lamp geometry and penumbra width recorded during the dark-room session.

### 1.4 Validating rendered realism against real images

Beyond space-specific simulators, general computer graphics research has also studied how to determine whether a rendered image is convincingly realistic when compared against real photographs, typically through structured user studies or quantitative image similarity metrics, as in the two-alternative forced-choice study by Kluge and Staadt [8]. This body of work is relevant background but is not the primary methodology adopted here: rather than asking whether a human observer finds the render convincing, Chapter 2 proposes asking whether the project's own automated reconstruction pipeline produces consistent results on the render and on the real photograph. This is a stricter and more directly relevant test for this application, since the pipeline, not a human viewer, is the actual downstream consumer of the images.

## Chapter 2

# Validation Methodology

### 2.1 Scope

This document addresses scene-level realism first: correct geometry, correct tether proxy appearance, and correct illumination (spectral color and irradiance, as defined in Chapter 1). Camera-level realism, sensor noise, and optical aberrations of the specific camera model in use, is treated as a future refinement and is not required for the first validation pass, since the reconstruction pipeline’s segmentation and tracking stages are expected to be more sensitive to illumination and contrast than to these secondary effects. This should be revisited if the first validation pass shows disagreement that scene-level factors cannot explain.

### 2.2 The core problem: no ground truth exists for the orbital condition

The central difficulty is that the condition of real interest, the tether photographed in orbit, cannot be photographed for real. There is no ground-truth image to compare against for that condition, which means realism cannot be validated directly where it matters most.

The proposed solution is to validate indirectly, through a condition that is one step removed from orbit but still has a real photographable reference: the laboratory dark enclosure. Figure 2.1 summarizes this two-tier strategy.

### 2.3 Comparison instrument: the reconstruction pipeline itself

Rather than relying on visual inspection or generic image-similarity metrics to decide whether the render "looks real enough," this methodology reuses the project’s own reconstruction pipeline as the comparison instrument. The logic is direct: the pipeline is the actual downstream consumer of these images in operational use, so agreement at the pipeline’s output is a more relevant realism criterion than agreement at the pixel level or to a human observer.

For a matched pair of images, one real and one simulated, of the same tether configuration, the following outputs are compared:

- **Segmentation mask:** coverage percentage and, where a pixel-level correspondence can be established, intersection-over-union between the real and simulated mask.
- **Endpoint detection:** position of the two detected tether ends, compared directly since both images represent the same physical configuration.
- **Path tracing:** coverage percentage and path length from the SmartWireTracker output.

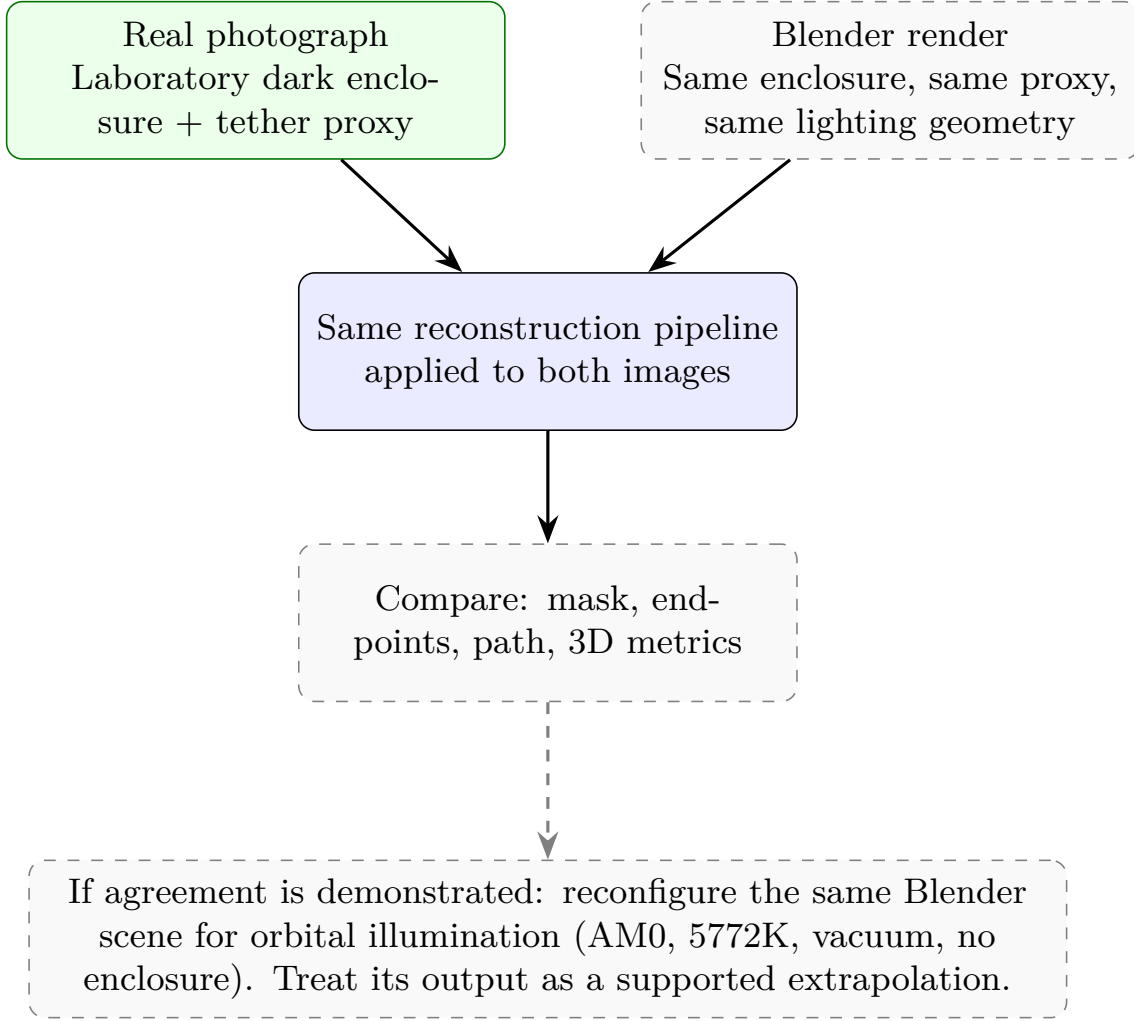


Figure 2.1: Two-tier validation strategy. Solid green boxes have a real reference available today. Dashed gray boxes are pending. The orbital-condition render (bottom) is only trusted to the extent that the laboratory-condition render (top) is shown to agree with its real photographic counterpart.

- **3D reconstruction metrics:** total length, straightness, and curvature, when both images are available as part of a stereo pair.

Close agreement across these outputs is the evidence required before the orbital-condition render is treated as informative rather than speculative.

## 2.4 Independent variables to control

To keep the comparison meaningful, the following must be matched between the real photograph and the Blender render as closely as physically and digitally possible:

- Tether proxy geometry and pose (same physical configuration, or a measured and replicated one).
- Camera intrinsic parameters (focal length, sensor size, field of view) matching the calibrated real camera.
- Camera extrinsic parameters (position and orientation relative to the tether) matching the real capture setup.
- Illumination geometry (light source position and direction relative to the tether and camera).

- Illumination spectral color and irradiance, per Chapter 1, matched to the real dark-enclosure light source for the laboratory-condition render, and set to AM0/5772K only for the orbital-condition render.

Pending: measure the real dark-enclosure light source's actual spectral color temperature and irradiance (if not already characterized in the Camera System Report) so the laboratory-condition Blender render matches the real light source rather than an assumed value.

## Chapter 3

# Experimental Setup

This chapter defines what needs to be built and captured to execute the methodology of Chapter 2. Most items in this chapter are still pending, with one exception documented in Section 3.1: a preliminary self-consistency check, distinct from the real-vs-render matched-pair comparison that remains the core pending experiment.

### 3.1 LINKU structure replica and preliminary self-consistency check

Ahead of the matched real/simulated pair comparison defined later in this chapter, the Blender scene (`SIMULATION/Simulation_DarkBox_2.blend`) was extended from a plain dark enclosure to a first replica of the LINKU deployment structure that the tether proxy is mounted on, with both stereo cameras (`Camera`, `Camera.001`) positioned inside it at a fixed 100 mm baseline. Figure 3.1 shows the scene from a general viewpoint, including the world background node setup; Figure 3.2 shows the tether as seen from each camera’s own perspective inside the enclosure.

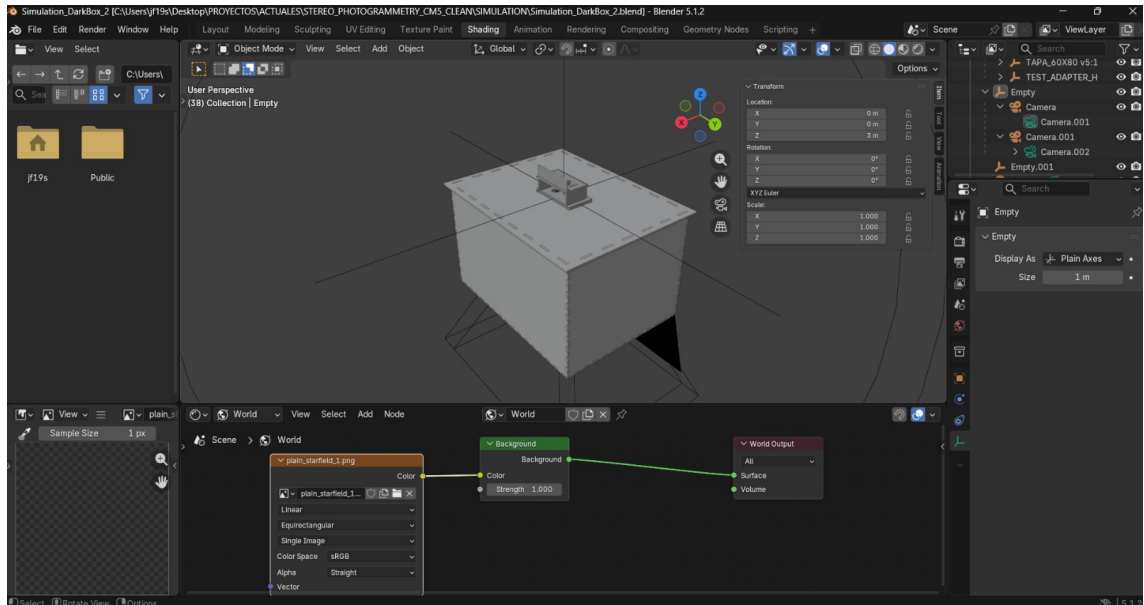


Figure 3.1: Blender replica of the LINKU deployment structure (`Simulation_DarkBox_2.blend`), general view. The stereo camera pair (`Camera`, `Camera.001`) is mounted at a fixed 100 mm baseline inside the enclosure; the World node setup uses an equirectangular background image rather than a flat color, matching the project’s established scene convention.

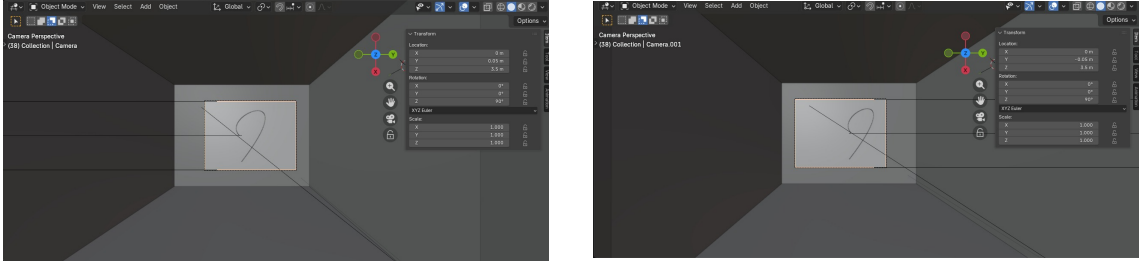


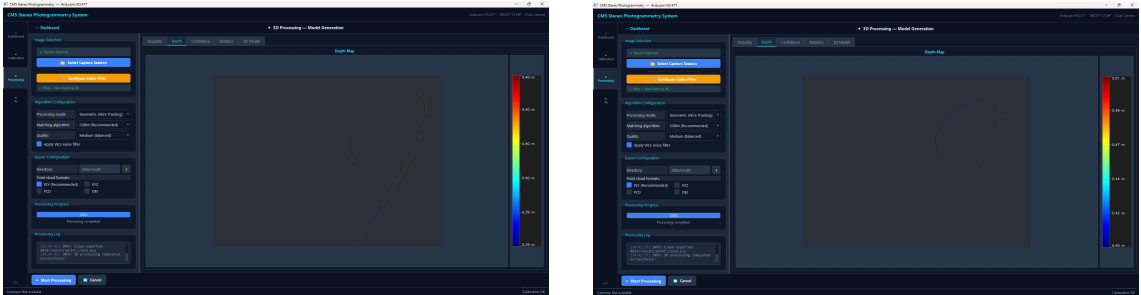
Figure 3.2: Tether proxy as seen from each stereo camera inside the Blender LINKU structure replica, **Camera** (left) and **Camera.001** (right), a 100 mm-baseline stereo pair at 3.5 m from the enclosure back wall.

With this scene, a preliminary check was run to confirm that the reconstruction pipeline recovers known geometry correctly when fed Blender-rendered stereo pairs, before attempting the harder real-photograph comparison. This check is a self-consistency test internal to Blender (synthetic images compared against a known synthetic displacement, not against a real photograph), using the Blender-derived calibration data/calibration/calibration\_data\_blender\_sim.json produced directly from the scene’s camera intrinsics and extrinsics rather than from checkerboard calibration. It should not be read as evidence toward the real-vs-render agreement question that Section 3.3 and Chapter 4 address; it validates the reconstruction pipeline and the Blender-derived calibration together, on ground truth that is exact by construction.

Two tether configurations were rendered and processed through the project’s full pipeline (segmentation, endpoint detection, geometric path tracking, disparity, and depth), viewed in the application’s Depth Map tab:

- **Configuration A:** tether close to a uniform 40 cm distance from the cameras. Reconstructed depth range: 0.39–0.40 m (Figure 3.3a).
- **Configuration B:** one tether endpoint displaced approximately 10 cm farther from the cameras relative to Configuration A. Reconstructed depth range: 0.40–0.51 m, an 11 cm spread (Figure 3.3b).

The reconstructed depth spread between the two configurations (11 cm) agrees with the known applied displacement (10 cm) to within 1 cm. This is a single preliminary case, not a systematic accuracy characterization across multiple poses and distances, but it supports proceeding to the matched real/simulated pair comparison with reasonable confidence that a disagreement found there would reflect a rendering or illumination realism gap rather than a defect in the pipeline or the Blender-derived calibration itself.



(a) Configuration A: 0.39–0.40 m

(b) Configuration B: 0.40–0.51 m

Figure 3.3: Depth Map tab output (application GUI) for the two known-displacement configurations described in Section 3.1. The colorbar legend (blue = near, red = far) and the neutral-gray background for pixels without valid depth are the fix applied to this view earlier in the same development session.

The underlying capture session and exported point cloud for this check were not yet pushed to the repository at the time of writing; only the GUI screenshots above were available. Push the `stereo_sim_*` session and `point_cloud.ply` for both configurations for reproducibility, following the project’s established synthetic-session convention.

## 3.2 Blender scene construction

Pending: build the Blender scene. Required elements: (1) tether proxy geometry matching the real proxy’s diameter and material appearance, (2) dark-enclosure interior geometry and matte-black material properties, (3) camera object with intrinsic parameters matching the calibrated real camera, (4) light source configured per Chapter 1: Blackbody shader at approximately 5772K, Sun strength set to the real light source’s measured irradiance for the laboratory-condition render, and to 1361 W/m<sup>2</sup> (AM0) for the orbital-condition render.

### 3.2.1 Chosen camera sensor and intrinsics

The flight camera sensor was decided during the master work plan’s sensor-decision task, using the depth-of-field and field-of-view comparison tool in `calculate_cam/camera_dof_visualizer.py`. The tool was extended to cover five candidate optical configurations (the existing IMX477 baseline, an Arducam Hawk-eye OV64A40, and three IMX708-based Arducam/Raspberry Pi modules with different lenses), evaluated by pixels occupied by a 1 mm-diameter thread at distances from 0.5 to 10 m. A sixth candidate, a Sony IMX296 global-shutter sensor proposed later for evaluation (Section 3.2.1.3), is included in the same comparison for completeness, with its two lens options.

#### 3.2.1.1 Governing equations

Every number in Table 3.1 comes from the same three equations below. The tool implements them as two small functions in `camera_dof_visualizer.py`: `pixels_per_mm_at_distance()` computes Equation 3.2, and `thread_pixels()` applies Equation 3.3 on top of it. All quantities are kept in millimetres internally; object distance  $u$  is converted from metres.

**Variables:**

- $f$  : lens focal length (mm)
- $u$  : object distance from the lens (mm)
- $R_h$  : sensor horizontal resolution (px)
- $W$  : sensor width (mm), so  $R_h/W$  is the sensor’s pixel density (px/mm), the inverse of pixel pitch
- $d$  : thread diameter (mm), 1 mm throughout this table
- $N_{px}$  : pixels occupied by the thread at distance  $u$

**1. Thin-lens magnification.** Standard thin-lens relation  $1/f = 1/v - 1/u$ , solved for the magnification  $m = v/u$ :

$$m(u) = \frac{f}{u - f} \quad (3.1)$$

**2. Object-space pixel density.** The sensor’s own pixel density ( $R_h/W$ , px/mm on the sensor) scales by the magnification to give pixels per mm of the real-world object at distance  $u$ :

$$\rho_{obj}(u) = \frac{R_h}{W} \cdot m(u) = \frac{R_h}{W} \cdot \frac{f}{u - f} \quad (3.2)$$

**3. Thread pixel occupancy.** The table’s thread-pixel columns are  $\rho_{obj}(u)$  scaled by the thread diameter:

$$N_{px}(u) = \rho_{obj}(u) \cdot d = \frac{R_h f d}{W (u - f)} \quad (3.3)$$

**4. 1 px-limit distance.** The table’s last column solves Equation 3.3 for  $u$  at  $N_{px} = 1$ ,  $d = 1$  mm:

$$u_{1px} = f \left( 1 + \frac{R_h}{W} \right) \quad (3.4)$$

Equation 3.3 is the one to read carefully before comparing sensors by eye: at  $u \gg f$  (every distance in this table, since  $u$  is 0.5 to 100 m and  $f$  is a few mm to 16 mm),  $u - f \approx u$  and  $N_{px}$  reduces to  $N_{px} \approx (R_h/W) \cdot f \cdot d/u$ . Total resolution ( $R_h \times R_v$ , megapixels) does not appear on its own; only  $R_h/W$  does, which is the sensor’s pixel density (equivalently, 1/pixel pitch), multiplied by focal length  $f$ . A sensor with far fewer total pixels can still win if its pixels are proportionally larger (higher  $R_h/W$  is not guaranteed by more megapixels, since  $W$  also varies between sensors) or its lens has longer focal length. Section 3.2.1.2 below works this through numerically for the two rows of Table 3.1 where this is least intuitive.

Table 3.1 reports the full comparison: sensor resolution, lens focal length (the parameter that actually separates telephoto from wide-angle, not aperture, which only sets depth of field and light gathering), horizontal field of view, thread pixel occupancy at four representative distances, and the distance at which a 1 mm-diameter thread is reduced to exactly 1 px, the practical geometric resolution limit for this detection task. All values computed directly from `camera_dof_visualizer.py`’s own `thread_pixels()` function (Equations 3.1–3.3), not re-derived by hand, so the table matches what the tool itself reports.

Table 3.1: Optical comparison of all candidate sensor/lens configurations for a 1 mm-diameter thread, computed with `calculate_cam/camera_dof_visualizer.py`. “1 px limit” is the distance at which the thread is reduced to exactly 1 pixel, the practical geometric detection limit. FOV H is computed as  $2 \arctan(W/2f)$  from each sensor’s own width, the same geometry the pixel equations use; vendor-advertised values differ (e.g.  $66^\circ$  for the B0308,  $120^\circ$  for the B0310) because product pages quote different conventions. Bold: chosen flight sensor. Italic: global-shutter candidate under evaluation (Section 3.2.1.3), not a flight-sensor decision.

| Sensor                         | Lens<br>(f)    | MP          | FOV H<br>(°) | Thread px (1 mm) at distance |             |             |               | 1 px limit<br>(distance) |
|--------------------------------|----------------|-------------|--------------|------------------------------|-------------|-------------|---------------|--------------------------|
|                                |                |             |              | 0.5 m                        | 2 m         | 10 m        | 100 m         |                          |
| <b>IMX477 (fitted today)</b>   | <b>8.0 mm</b>  | <b>12.3</b> | <b>42.9</b>  | <b>10.49</b>                 | <b>2.59</b> | <b>0.52</b> | <b>0.0516</b> | <b>5.17 m</b>            |
| IMX477 with the 6 mm lens      | 6.0 mm         | 12.3        | 55.3         | 7.84                         | 1.94        | 0.39        | 0.0387        | 3.88 m                   |
| Hawk-eye OV64A40               | 5.1 mm         | 63.6        | 71.9         | 12.75                        | 3.16        | 0.63        | 0.0631        | 6.31 m                   |
| IMX708 Wide (B0310)            | 2.87 mm        | 11.9        | 96.7         | 4.12                         | 1.03        | 0.21        | 0.0205        | 2.05 m                   |
| IMX708 Wide 102°               | 2.75 mm        | 11.9        | 99.1         | 3.95                         | 0.98        | 0.20        | 0.0196        | 1.97 m                   |
| <b>IMX708 Standard (B0308)</b> | <b>4.74 mm</b> | <b>11.9</b> | <b>68.5</b>  | <b>6.84</b>                  | <b>1.70</b> | <b>0.34</b> | <b>0.0339</b> | <b>3.39 m</b>            |
| <i>IMX296 global shutter</i>   | <i>16.0 mm</i> | <i>1.6</i>  | <i>17.7</i>  | <i>9.69</i>                  | <i>2.36</i> | <i>0.47</i> | <i>0.0469</i> | <i>4.70 m</i>            |
| <i>IMX296 global shutter</i>   | <i>8.0 mm</i>  | <i>1.6</i>  | <i>34.5</i>  | <i>4.76</i>                  | <i>1.18</i> | <i>0.23</i> | <i>0.0234</i> | <i>2.35 m</i>            |
| <i>IMX296 global shutter</i>   | <i>6.0 mm</i>  | <i>1.6</i>  | <i>45.0</i>  | <i>3.56</i>                  | <i>0.88</i> | <i>0.18</i> | <i>0.0176</i> | <i>1.76 m</i>            |

**Correction, and what it affects.** The first row of Table 3.1 previously described the IMX477 with the 6 mm lens as the current configuration. That is wrong: the stereo pair in use carries the Arducam C1508ZM04, 8 mm F1.6–F16, as recorded in Chapter 5 of the D5 Camera System Report. Both rows are now shown, since the project owns both lenses. The 8 mm row was computed with the same equations and the same tool as every other row. The comparison in Section 3.2.1.2 between the chosen B0308 and “the current IMX477 baseline” was written against the 6 mm figures and is left as written, because the B0308 decision also rested on field of view, downlink budget and existing mounting CAD; the pixel-resolution part of that comparison does need to be re-read against the 8 mm row.

Pending: re-examine the B0308 selection argument against the fitted 8 mm IMX477 baseline (10.49 px at 0.5 m, 1 px limit at 5.17 m) rather than the 6 mm one (7.84 px, 3.88 m). The gap to the B0308’s 6.84 px is wider than the text currently states.

**Two details behind the table.** The three IMX296 rows pair that sensor with each of the project’s three interchangeable lenses, so the cost of a lens swap can be read off directly; the 8 mm row is the operationally relevant one, since that is the lens the project actually has fitted. All three use the comparator’s stored sensor width of 4.97 mm, where deriving it from the Sony datasheet

as 1456 pixels at a 3.45  $\mu\text{m}$  pitch gives 5.023 mm. The stored value is kept so every row stays mutually consistent; the 1% difference it causes in the IMX296 thread-pixel figures is immaterial here.

On the convention noted in the caption: the stored `fov_h_deg` values are vendor figures for assorted optical formats, and the worst case is the 6 mm lens, stored as 65° where the geometry gives 55.3°, because 65° is close to the IMX477's 66.4° *diagonal* rather than its horizontal. No column of this table is affected, since Equation 3.3 uses  $R_h$ ,  $W$  and  $f$  and never a stored field of view.

At 100 m, none of the nine configurations keeps the 1 mm thread above 1 px (values range from 0.018 px to 0.063 px per thread, roughly two orders of magnitude below the geometric detection limit); at that distance detection depends on the sub-pixel radiometric regime (specular reflection against a dark background), treated in the companion deliverable *Dark-Room Solar Illumination Specification for Tether Imaging*, not on geometric resolution. The 1 px-limit distance column is therefore the more directly useful number for comparing configurations at the scales relevant to this mission.

### 3.2.1.2 Worked example: why a 1.6 MP sensor is not simply outclassed by a 11.9 MP one

The least intuitive pair in Table 3.1 is the IMX296 with the 6 mm wide lens (1.6 MP) against the chosen IMX708 Standard / B0308 (11.9 MP): at 100 m the IMX296 reads 0.0176 px against the B0308's 0.0339 px, roughly half, despite the B0308 having 7.4× the total pixel count. Equation 3.3 resolves this by substituting both sensors' values at  $u = 100\,000$  mm,  $d = 1$  mm:

**IMX296 + 6 mm:**  $R_h = 1456$  px,  $W = 4.97$  mm,  $f = 6.0$  mm

$$\begin{aligned}\rho_{\text{sensor}} &= R_h/W = 1456/4.97 = 293.0 \text{ px/mm} \quad (\text{pixel pitch } 1/\rho_{\text{sensor}} = 3.41 \text{ } \mu\text{m}) \\ m &= f/(u - f) = 6.0/99994.0 = 6.000 \times 10^{-5} \\ N_{\text{px}} &= \rho_{\text{sensor}} \cdot m \cdot d = 293.0 \times 6.000 \times 10^{-5} \times 1 = 0.0176 \text{ px}\end{aligned}$$

**IMX708 Standard (B0308):**  $R_h = 4608$  px,  $W = 6.45$  mm,  $f = 4.74$  mm

$$\begin{aligned}\rho_{\text{sensor}} &= 4608/6.45 = 714.4 \text{ px/mm} \quad (\text{pixel pitch } 1.40 \text{ } \mu\text{m}) \\ m &= 4.74/99995.26 = 4.740 \times 10^{-5} \\ N_{\text{px}} &= 714.4 \times 4.740 \times 10^{-5} \times 1 = 0.0339 \text{ px}\end{aligned}$$

Taking the ratio directly from Equation 3.3 at  $u \gg f$  (so  $m \approx f/u$  for both, and  $u$  cancels):

$$\frac{N_{\text{px}}^{\text{IMX296}}}{N_{\text{px}}^{\text{B0308}}} \approx \frac{\rho_{\text{sensor}}^{\text{IMX296}}}{\rho_{\text{sensor}}^{\text{B0308}}} \times \frac{f^{\text{IMX296}}}{f^{\text{B0308}}} = \frac{293.0}{714.4} \times \frac{6.0}{4.74} = 0.410 \times 1.266 = 0.519$$

which matches the 0.0176/0.0339 = 0.519 read off the table. The 7.4× total-pixel-count gap barely enters: what matters is  $R_h/W$  (pixel density on the sensor, the inverse of pixel pitch), not  $R_h \times R_v$  (total megapixels). The IMX296's pixel pitch (3.41  $\mu\text{m}$ ) is 2.44× larger than the B0308's (1.40  $\mu\text{m}$ ), so despite having 3.16× fewer horizontal pixels (1456 vs 4608) it only loses a factor of 0.410 in sensor-side pixel density, not 3.16× or 7.4×. Paired with a slightly longer focal length in this comparison (6.0 mm vs 4.74 mm, a 1.266× factor in the IMX296's favor, since this 6 mm lens is the IMX477's lens reused for a like-for-like FOV comparison, not the IMX296's own quoted telephoto), the two factors combine to 0.519, not the much smaller ratio a megapixel-only comparison would suggest. The same mechanism, with a 16 mm instead of 6 mm focal length, is why the IMX296's telephoto configuration (Table 3.1, Section 3.2.1.3) overtakes the B0308 outright rather than merely approaching it.

A second, purely geometric check gives the same answer without any lens formula. A thread of diameter  $d$  at distance  $u$  subtends an angle  $\theta \approx d/u$ , and a sensor with  $R_h$  horizontal pixels spread over a horizontal field of view  $\text{FOV}_h$  places a fraction  $\theta/\text{FOV}_h$  of its pixels on it, so  $N_{\text{px}} \approx R_h \theta/\text{FOV}_h$ . What matters is pixels per degree,  $R_h/\text{FOV}_h$ : the IMX296 with the 6 mm lens has 1456/45  $\approx 32$  px/°, the B0308 has 4608/68.5  $\approx 67$  px/°, a ratio of 0.48, about 8% below

the 0.52 of the full equation, the gap being the small-angle approximation. The total pixel count is a poor proxy because it ignores how wide the lens spreads those pixels. This approximation is looser for very wide fields of view (the small-angle form breaks down near  $100^\circ$ ), so Equation 3.3 remains the reference; for the fields used in Table 3.1 the two methods agree to within about 10%, with the pixel values in the table taken from Equation 3.3.

The result: the IMX708 module with its standard  $66^\circ$  horizontal field-of-view lens, fixed-focus variant, SKU B0308, was chosen. At every tested distance it matched the current IMX477 baseline’s tether pixel resolution within a few percent (for example, 6.8 px versus 7.8 px at 0.5 m, both crossing below 1 px per thread around 4 to 5 m), while the two wider IMX708 lens options ( $102^\circ$  and  $120^\circ$  horizontal FOV) lost resolvable pixels on the thread noticeably earlier, around 2 m. The Hawk-eye OV64A40 resolved the thread furthest (to roughly 8 to 10 m) but was not selected, since its 64 MP output is difficult to reconcile with the mission’s downlink budget (see the communications link budget in the APSCO deliverable), and it is a non-standard sensor not covered by the mechanical mounting CAD already produced for the IMX708 module. The fixed-focus B0308 variant was preferred over its autofocus siblings (B0306, B0312, same optics) because a fixed lens has no moving focus mechanism to survive launch vibration or vacuum operation.

### 3.2.1.3 Global-shutter candidate: where it actually stands

The IMX296 global-shutter sensor (proposed later, Section 3.1 documents its own geometric validation separately) is not a replacement candidate for the B0308 flight-sensor decision above, which stands as made. It is being evaluated specifically for its freedom from rolling-shutter distortion during the moving-tether experiment. Table 3.1 shows it is a stronger optical performer than its 1.6 MP figure alone suggests: paired with the 16 mm telephoto lens it comes with, it resolves the thread to a greater distance (4.70 m) than the chosen B0308 (3.39 m) or the IMX477 baseline (3.88 m), despite about a seventh of their pixel count, because the narrow  $17.7^\circ$  field of view concentrates more of the sensor’s pixels per degree of scene. The trade-off is exactly that narrow field of view (about a quarter of the B0308’s horizontal coverage,  $17.7^\circ$  against  $68.5^\circ$ ) and, at this lens’s fast f/1.4 end, a correspondingly shallow depth of field, so a global-shutter design demands tighter aiming and distance control than the wide-FOV flight sensor. Paired instead with a 6 mm wide lens matching the IMX477’s own, the IMX296 underperforms every other candidate (1.76 m), since about a seventh of the pixel count is no longer compensated by a narrower field of view. The IMX296 is therefore only competitive in its telephoto configuration, and that configuration’s narrow FOV is a real operational cost, not a free upgrade.

The B0308 unit itself has not been purchased yet; procurement is realistically expected next year, tracked as its own item in the master work plan. Until then, near-term physical work, including the laboratory-condition renders and real photographs this chapter defines, continues with the IMX477 hardware already available, which Section 3.2.2 below addresses directly. Table 3.2 lists the B0308’s intrinsics regardless, since they are the values the Blender camera object should be configured with once the sensor is purchased and this validation is repeated against it.

Table 3.2: Chosen camera intrinsics, Arducam/Raspberry Pi 12MP IMX708 Standard, fixed focus (SKU B0308).

| Parameter         | Value                                            |
|-------------------|--------------------------------------------------|
| Sensor            | Sony IMX708                                      |
| Sensor size       | $6.45 \times 3.63$ mm                            |
| Max resolution    | $4608 \times 2592$ px (11.9 MP)                  |
| Pixel size        | $1.4 \mu\text{m} \times 1.4 \mu\text{m}$         |
| Lens focal length | 4.74 mm                                          |
| Lens aperture     | f/1.8, fixed                                     |
| Field of view     | $66^\circ$ (H) / $75^\circ$ (D) / $41^\circ$ (V) |
| Focus             | Fixed, manufacturer-stated 1.5 m to infinity     |

The 1.5 m minimum object distance above is taken from the fixed-focus B0308 product page directly; it was not independently bench-verified. Confirm once the physical unit is in hand, before relying on it for the eventual B0308 laboratory-condition capture.

### 3.2.2 Near-term substitution: laboratory work proceeds with the IMX477

Since the B0308 is not yet in hand, the laboratory photograph capture and matched-pair generation described later in this chapter use the IMX477 hardware already installed and previously exercised (for example, the reference pair `data/captures/stereo_20260213_140535` used in Section 3.3). This is a reasonable substitution for this validation’s purpose: Section 3.2.1 already established that the B0308 and the IMX477 resolve the reference thread to within a few percent of each other at the same distance, so agreement (or disagreement) found between a Blender render and a real IMX477 photograph is expected to transfer to the B0308 case once it is available, rather than being an artifact of one specific sensor.

This substitution also removes the practical risk noted above: the IMX477’s minimum focus distance is on the order of tens of centimeters, comfortably inside the dark enclosure’s working distance, unlike the B0308’s stated 1.5 m. The Blender camera object for the laboratory-condition render should therefore use the IMX477 intrinsics (matching the real calibrated camera, as already required earlier in this section) for now, and be reconfigured to the B0308 intrinsics in Table 3.2 once that sensor is purchased and this validation is repeated against it.

### 3.2.3 Blender’s own configuration reference for the Sun light

Blender’s own documentation, rather than a third-party tutorial, confirms two of the settings already used in the project’s scene file. The Sun light object exposes an **energy** property described directly in the Blender Python API reference as “Sunlight strength in watts per meter squared ( $\text{W}/\text{m}^2$ )”, unbounded range, default 10.0 [9], which is the same irradiance unit discussed in Chapter 1 and confirms the value can be set directly with no conversion. The same object exposes an **angle** property, described as the “Angular diameter of the Sun as seen from the Earth”, range 0 to  $\pi$  radians, default 0.00918043 rad ( $\approx 0.526^\circ$ ) [9]. This matches, to three significant figures, the real solar angular diameter ( $\approx 0.53^\circ$ ) already set on the Sun object in `SIMULATION/Simulation_DarkBox.blend`, which is therefore confirmed correct against Blender’s own default rather than only against the external astronomical figure used when it was first set.

Blender’s manual is explicit that scene-linear light values in Cycles are unbounded (0 to infinity) and only compressed to a displayable image by the View Transform chosen in Color Management, not by the light values themselves [10]. The default View Transform in Blender 5.x, AgX, is described in the manual as offering approximately 16.5 stops of dynamic range and deliberately desaturating strongly overexposed colors rather than clipping them, so that bright regions roll off toward white the way a real camera’s sensor does instead of producing flat, saturated color blocks [10]. This is the mechanism, native to Blender and not a custom workaround, that directly satisfies the requirement that the rendered light must not appear saturated: keeping View Transform set to AgX (the default) rather than Standard is the concrete action to take when the scene is rendered, and the Exposure slider (output equals the rendered value multiplied by  $2^{\text{exposure}}$ ) is the correct control to fine-tune brightness afterward without reintroducing clipping.

## 3.3 Laboratory photograph capture

A reference stereo pair has been selected from the existing capture archive: the tether enters the frame from above the camera and falls toward the background, chosen because this configuration is simple to reproduce and reason about (no self-crossing, no ambiguous topology) for a first matched-pair test.

- Left image: `data/captures/stereo_20260213_140535/left.jpg`
- Right image: `data/captures/stereo_20260213_140535/right.jpg`
- Metadata: `data/captures/stereo_20260213_140535/metadata.json` (capture resolution  $1920 \times 1440$ , baseline 100 mm, synchronization quality "EXCELENTE", 10.4 ms time difference between cameras)

Pending: this pair was captured before the current calibration was invalidated (Chapter 7 recalibration item, Camera System Report), so it is usable as a 2D reference for mask/endpoint/path comparison but not for 3D reconstruction comparison. Also pending: document the physical pose of the tether proxy for this specific capture (approach angle, distance from camera, support/attachment point) so it can be replicated in the Blender scene.

### 3.4 Matched-pair generation

Pending: for each real photograph captured above, render the corresponding Blender scene from the same camera position, orientation, and tether pose, producing one matched real/simulated pair per configuration tested. A minimum of 3 to 5 matched pairs, covering different tether poses (straight, curved, and one closer to a self-crossing configuration if feasible), is recommended before drawing conclusions from a single case.

### 3.5 Orbital-condition render generation

Pending, and dependent on satisfactory agreement in the laboratory-condition comparison (Chapter 4): reconfigure the validated Blender scene to represent orbital conditions (vacuum, no enclosure, AM0 solar illumination, unobstructed star-field or black background) and render the tether under representative deployment geometries.

## Chapter 4

# Evaluation and Reporting Plan

This chapter defines how results will be evaluated and reported once the experiments of Chapter 3 are executed. No results exist yet; this chapter defines the criteria that will be applied to them.

### 4.1 Reporting format

For each matched real/simulated pair, the following will be reported side by side:

- The real photograph and the Blender render, for visual reference.
- The segmentation mask obtained from each, with coverage percentage.
- The detected endpoints from each, with pixel-distance between the real and simulated endpoint positions.
- The traced path from each, with coverage percentage and path length.
- A qualitative note on any visible discrepancy and a hypothesis for its cause (for example: material reflectance mismatch, light source position error, or camera intrinsic mismatch).

### 4.2 Interpretation criteria

Pending decision, to be set before running the experiment rather than after seeing the results, to avoid post-hoc rationalization: define the quantitative agreement threshold (for example, endpoint position error below N pixels, mask coverage within X percentage points) below which the laboratory-condition render is considered validated and the orbital-condition extrapolation is considered supported.

### 4.3 Outcome paths

Two outcomes are possible once the laboratory-condition comparison is complete:

- **Agreement within the defined threshold:** proceed to generate and use orbital-condition renders, with the explicit caveat that confidence is inherited from laboratory-scale agreement and has not been independently verified for the orbital condition itself.
- **Disagreement beyond the defined threshold:** do not proceed to orbital-condition renders until the source of disagreement is identified and corrected. Likely candidates, in rough order of expected impact, are light source spectral or irradiance mismatch, tether material reflectance mismatch, and camera intrinsic parameter mismatch.

## Chapter 5

# Conclusions and Status

This document defines a concrete, literature-grounded methodology for establishing whether Blender-rendered images of the LINKU tether can be trusted as a proxy for photographs that cannot yet be taken physically, most importantly the orbital illumination condition. The methodology follows the precedent set by SISPO, validating simulator realism through direct comparison against real reference imagery rather than through visual inspection, and adapts it to this project by using the reconstruction pipeline itself, rather than a human observer, as the comparison instrument.

The literature review identified the correct physical values to use for solar illumination in Blender: a Blackbody shader at approximately 5772K for spectral color, and a Sun strength of approximately 1361 W/m<sup>2</sup> (AM0, extraterrestrial solar constant) for orbital-condition irradiance, as opposed to the lower ground-level value assumed by default in most Blender tutorials.

### 5.1 Current status

- Literature review: complete.
- Methodology definition: complete.
- Blender scene construction: pending.
- Laboratory photograph capture: pending.
- Matched-pair comparison: pending.
- Orbital-condition render generation: pending, and dependent on the laboratory-condition result.

### 5.2 Next steps

1. Measure or confirm the real dark-enclosure light source's spectral and irradiance characteristics.
2. Build the Blender scene replicating the laboratory dark enclosure.
3. Capture the reference real photographs.
4. Render the matched simulated counterparts and run the comparison defined in Chapter 2.
5. Decide, using the interpretation criteria of Chapter 4, whether to proceed to orbital-condition renders.

# Bibliography

- [1] M. Pajusalu, I. Iakubivskyi, G. J. Schwarzkopf, O. Knuuttila, T. Väisänen, M. Bühner, M. F. Palos, H. Teras, G. Le Bonhomme, J. Praks, and A. Slavinskis, “SISPO: Space imaging simulator for proximity operations,” *PLOS ONE*, vol. 17, no. 3, p. e0263882, 2022. <https://doi.org/10.1371/journal.pone.0263882>.
- [2] Blendergrid, “Using physically correct brightness in cycles.” <https://blendergrid.com/articles/cycles-physically-correct-brightness>. Accessed 2026.
- [3] M. Pugliatti, C. Buonagura, and F. Topputo, “CORTO: The celestial object rendering tool at DART lab,” *Sensors*, vol. 23, no. 23, p. 9595, 2023.
- [4] M. Denninger, M. Sundermeyer, D. Winkelbauer, Y. Zidan, D. Olefir, M. Elbadrawy, A. Lodhi, and H. Katam, “BlenderProc.” arXiv preprint arXiv:1911.01911, German Aerospace Center (DLR). <https://arxiv.org/abs/1911.01911>, 2019.
- [5] J. G. Singla, H. B. Patel, and N. Dube, “Evaluation of image simulation open source solutions for simulation of synthetic images in lunar environment.” arXiv preprint arXiv:2604.22296, Space Applications Centre, ISRO. <https://arxiv.org/abs/2604.22296>, 2026.
- [6] M. Meftah, L. Damé, D. Bolsée, A. Hauchecorne, N. Pereira, D. Sluse, G. Cessateur, A. Irbah, J. Bureau, M. Weber, K. Bramstedt, T. Hilbig, R. Thiéblemont, M. Marchand, F. Lefèvre, A. Sarkissian, and S. Bekki, “SOLAR-ISS: A new reference spectrum based on SOLAR/SOLSPEC observations,” *Astronomy & Astrophysics*, vol. 611, p. A1, 2018. <https://doi.org/10.1051/0004-6361/201731316>.
- [7] G. Thuillier, P. Zhu, M. Snow, P. Zhang, and X. Ye, “Characteristics of solar-irradiance spectra from measurements, modeling, and theoretical approach,” *Light: Science & Applications*, vol. 11, p. 79, 2022. <https://doi.org/10.1038/s41377-022-00750-7>.
- [8] S. Kluge and O. Staadt, “Assessing photorealism of rendered objects in real-world images: A transparent and reproducible user study.” arXiv preprint arXiv:2407.01767. <https://arxiv.org/abs/2407.01767>, 2024.
- [9] Blender Foundation, “SunLight(Light) — blender python api reference.” <https://docs.blender.org/api/current/bpy.types.SunLight.html>. Accessed 2026.
- [10] Blender Foundation, “Color management — blender manual.” [https://docs.blender.org/manual/en/latest/render/color\\_management.html](https://docs.blender.org/manual/en/latest/render/color_management.html). Accessed 2026.